MeteoCity

Monitoring City Weather — Milan & Rome

Politecnico di Milano · Software Engineering for Geoinformatics

Academic Year 2025/26 · Group 9

Ayman Abdelgadir

Mobina Faraji

Hangyun Wang

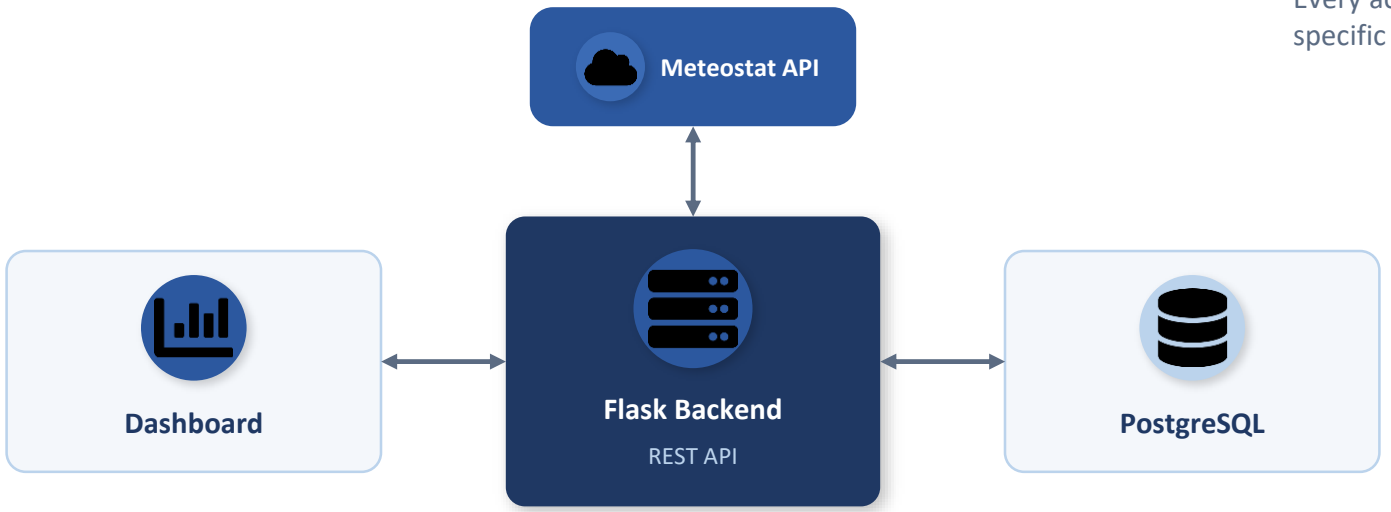
Supervised By :

Dr.Danilo Ardagna Dr.Elisabetta Di Nitto

June 2026

Flask — The System's Orchestrator

A lightweight Flask REST API server sits between the dashboard, the database, and Meteostat.



Only Flask ever talks to Meteostat directly — the dashboard never does.

What is an API Endpoint?

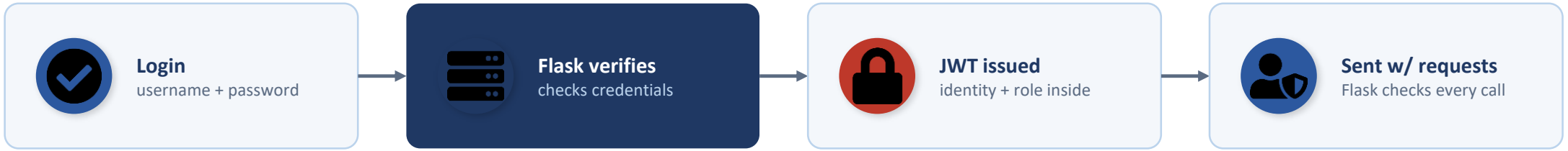
Every action the user takes (login, query, export...) becomes one HTTP request to a specific URL, answered with a JSON response.

Endpoint	Purpose
`POST /auth/login`	Authenticate user, return JWT
`GET /api/stations`	Weather station metadata
`GET /api/observations`	Hourly weather data
`GET /api/statistics`	Stats summary (Gov. only)
`GET /api/export_csv`	Download filtered data

Flask was chosen as a lightweight WSGI framework — minimal overhead, explicit routing, well suited to a clean REST API backend.

JWT — Securing Who Sees What

At login, Flask issues a signed token carrying the user's identity and role — sent with every future request, like a wristband proving access.



The token carries the user's role, never the password — the payload is verified on every subsequent call.

Role-Based Access Control

Role	Map / Query	Trend Analysis	Export CSV	Statistics
Government Official	✓	✓	✓	✓
NGO	✓	✓	✓	✗
Student	✓	✗	✓	✗

Getting Real Weather Data In

One backend module bridges Meteostat and PostgreSQL — using a single shared RapidAPI key configured for the deployment, not per dashboard user.

Mock Mode

No RapidAPI key needed — synthetic but realistic data lets the whole system run and demo instantly.

Live Mode

Real hourly observations fetched from Meteostat via one shared RapidAPI key in backend/.env.

30-Day Constraint

Meteostat caps requests at 30 days; longer ranges are auto-supplemented with synthetic data.

4 Hardcoded Stations

Milano Malpensa · Milano Linate · Roma Fiumicino · Roma Ciampino — two stations per city.

Database

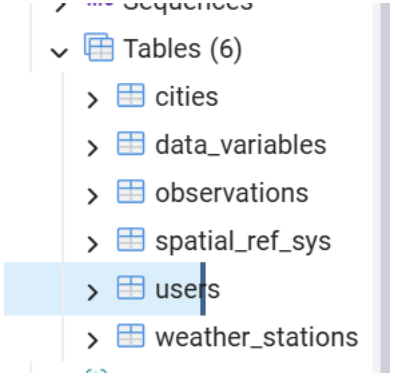
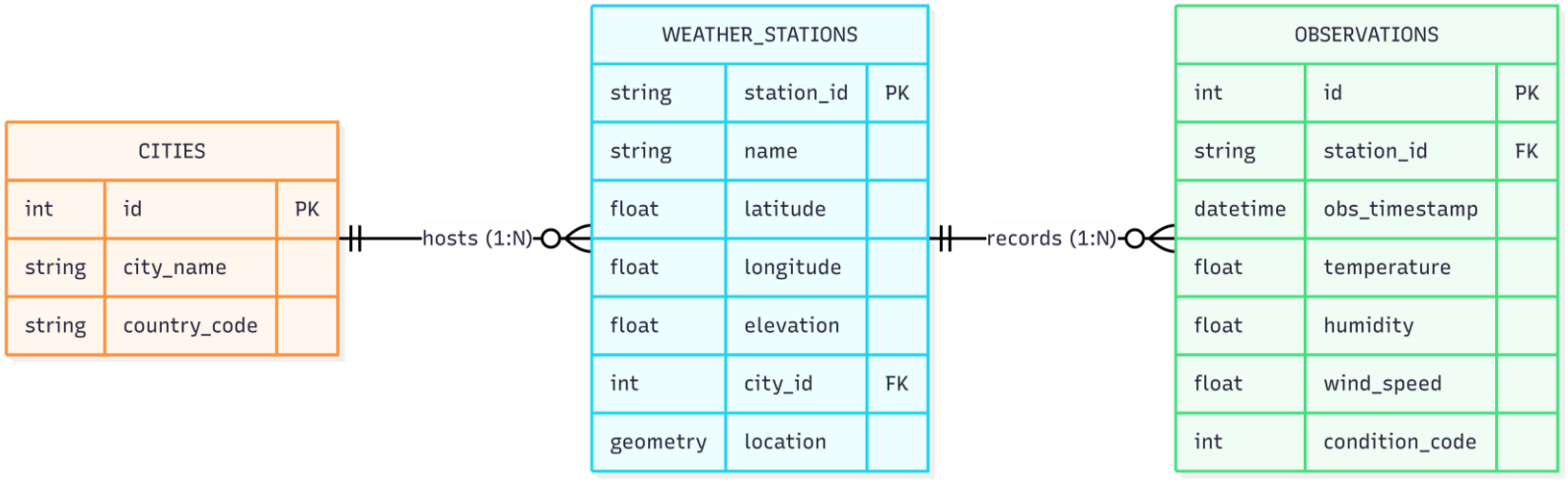


Table Name	Model Class	Description	Core Columns
users	User	Stores system user accounts, credentials, and access roles.	id, username, password_hash, role, created_at
cities	City	Stores city information and country codes where stations are located.	id, city_name, country_code
weather_stations	WeatherStation	Stores weather station metadata and geographical coordinates.	station_id, name, latitude, longitude, elevation, location, city_id
observations	Observation	Stores the actual weather metrics captured over time.	id, station_id, obs_timestamp, temperature, humidity, wind_speed, condition_code
data_variables	DataVariable	Stores the data dictionary/definitions for weather metrics (e.g., units).	id, variable_name, unit, description

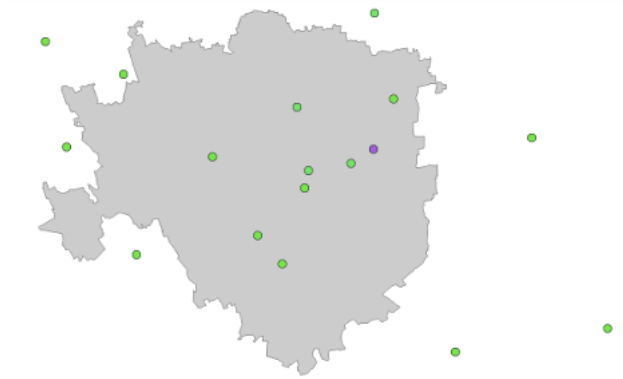
Tables

USERS		
int	id	PK
string	username	
string	password_hash	
string	role	
datetime	created_at	

DATA_VARIABLES		
int	id	PK
string	variable_name	
string	unit	
text	description	



GIS Spatial Extension



Integrated the **GeoAlchemy2** extension to embed true PostGIS functionalities directly inside the Flask-SQLAlchemy object-relational model.

```python
class WeatherStation(db.Model):
    __tablename__ = "weather_stations"

    station_id = db.Column(db.String(20), primary_key=True)
    name = db.Column(db.String(150), nullable=False)
    latitude = db.Column(db.Float, nullable=False)
    longitude = db.Column(db.Float, nullable=False)
    elevation = db.Column(db.Float)
    location = db.Column(Geometry(geometry_type="POINT", srid=4326))
    city_id = db.Column(db.Integer, db.ForeignKey("cities.id"), nullable=False)
```

Technical Advantages & Business Value:

SRID 4326 Alignment: Enforces global WGS 84 ellipsoid coordinate mapping natively.
Server-Side Spatial Compute: Executes geodesic distance, buffer creation, and containment logic natively within the SQL engine.
Workload Optimization: Shifts heavy geospatial computational math from client browsers to the optimized database backend.

Performance & Constraints

Unique Constraint

Implements a database-level constraint for atomic temporal checks. Under heavy concurrent traffic or data re-ingestion retries, any spatio-temporal overlapping records are strictly aborted, guaranteeing zero dataset contamination.

Composite Index

A B-Tree composite index on **(station_id, obs_timestamp)** cuts time complexity .This avoids full-table scans, enabling **sub-second retrieval** across millions of chronological observation rows to power fluid map rendering.

```
__table_args__ = (
    UniqueConstraint("station_id", "obs_timestamp", name="uq_station_timestamp"),
    Index("ix_obs_station_time", "station_id", "obs_timestamp"),
)
```

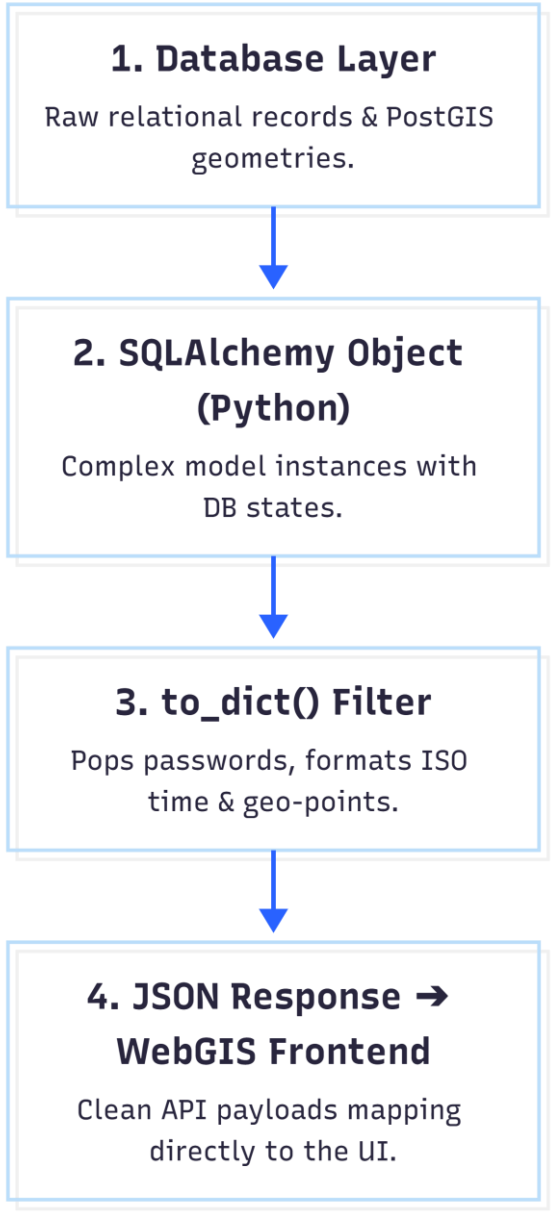
Bridging Backend to Web: To_dict()

Data Serialization Translates complex Python SQLAlchemy database objects into standard, browser-readable JSON/dictionary formats.

Security Shield Establishes an egress data "firewall" to actively filter out sensitive fields (e.g., password_hash), exposing only necessary metrics

WebGIS Integration Formats timestamps via .isoformat() and extracts coordinate pairs into clean payloads for direct frontend map framework rendering (Leaflet/Mapbox).

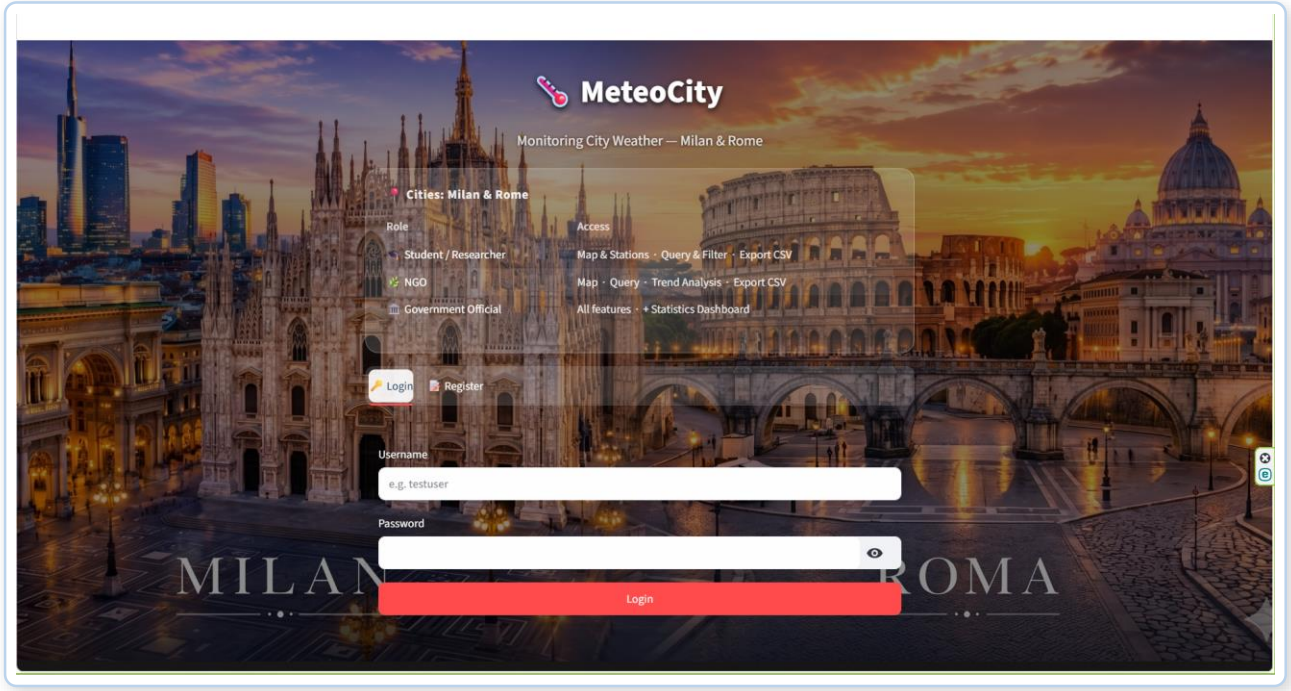
System Decoupling Acts as a dedicated data-layer translator, ensuring interface formats remain unified, compact, and perfectly optimized for JavaScript parsing



DASHBOARD · PART 1 OF 4

The User-Facing Layer

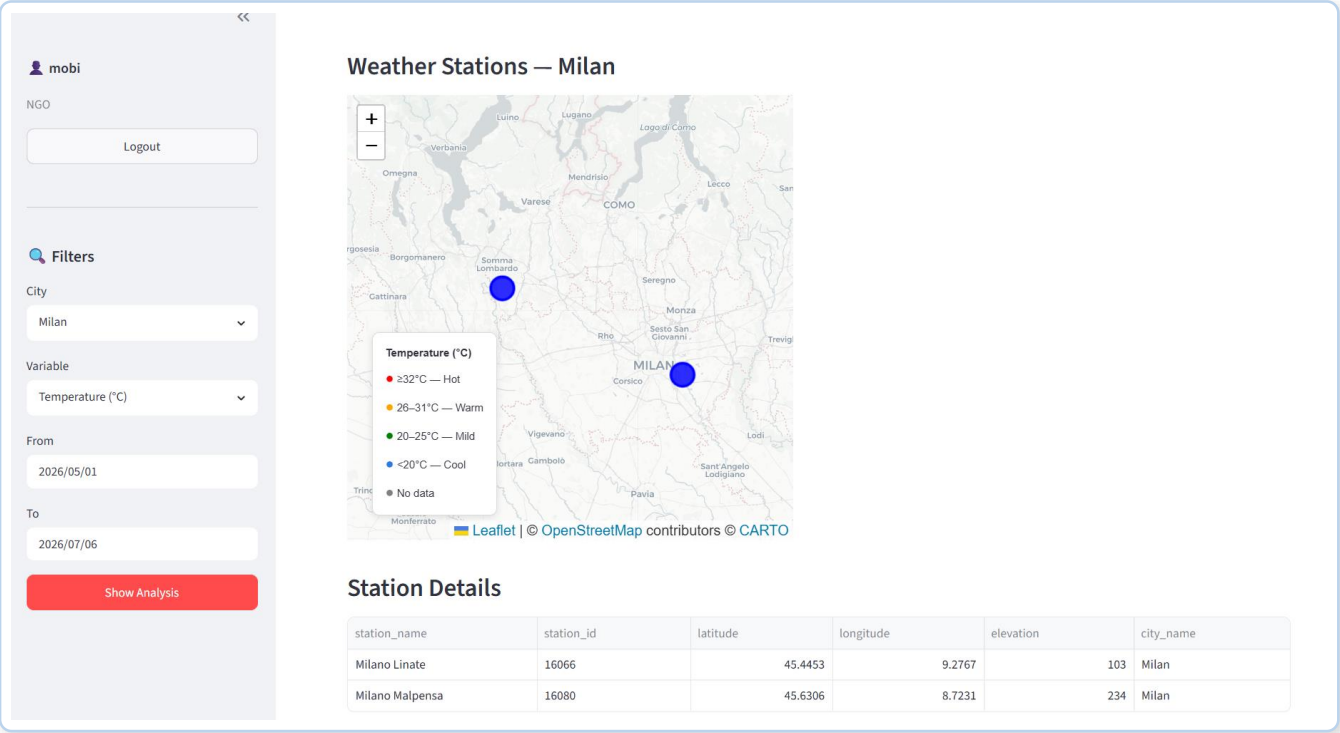
Login page explains available access levels before the user enters the dashboard.



User Action	Dashboard Response
Login / Register	creates an authenticated session
Select role	controls visible sections
Navigate tabs	map, trends, export views

Map & Filters — Queries into Context

The main view combines city/date/variable filters with a station map and a station metadata table.



What the Map View Provides

Filter-Driven Loading

City, variable, and date range are chosen before data is requested.

Geographic Station View

Stations are plotted on an interactive Leaflet map from backend coordinates.

Station Metadata

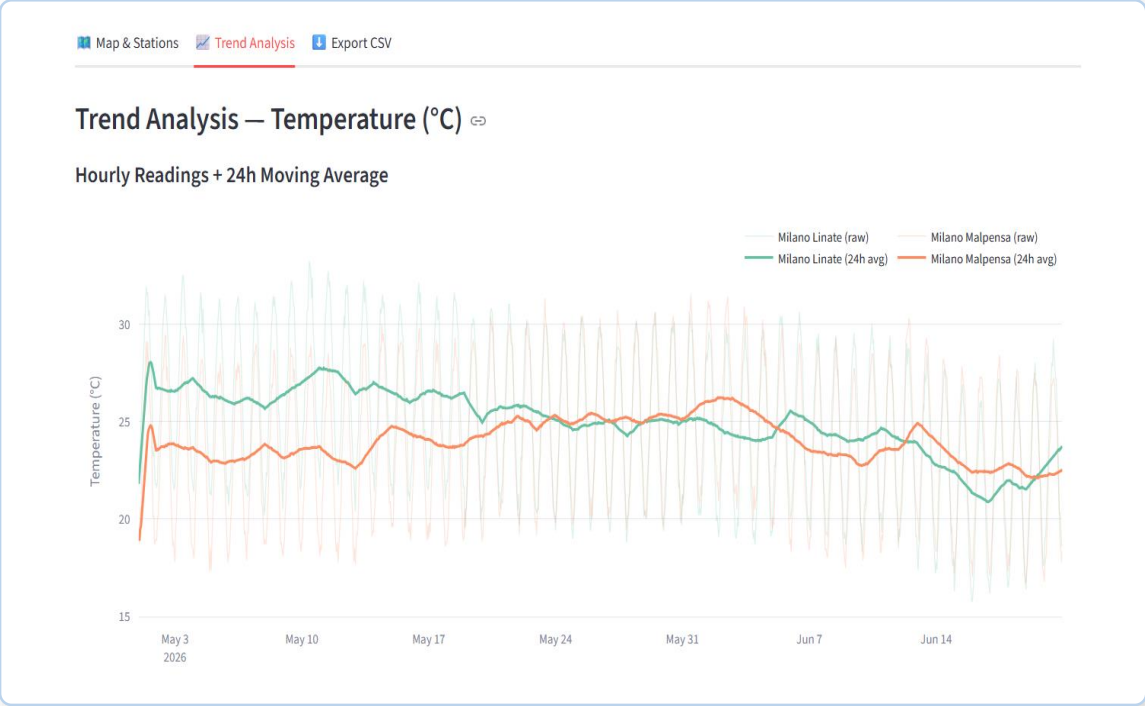
The table below gives id, name, coordinates, elevation, and city.

Example shown: Milan selected, temperature variable, and the two Milan stations returned by the backend.

Trend Analysis — From Data to Insight

After loading the filtered observations, the dashboard creates interactive charts for comparison.

Hourly Trend



24h moving average makes hourly readings easier to compare across stations.

Weekly & Distribution

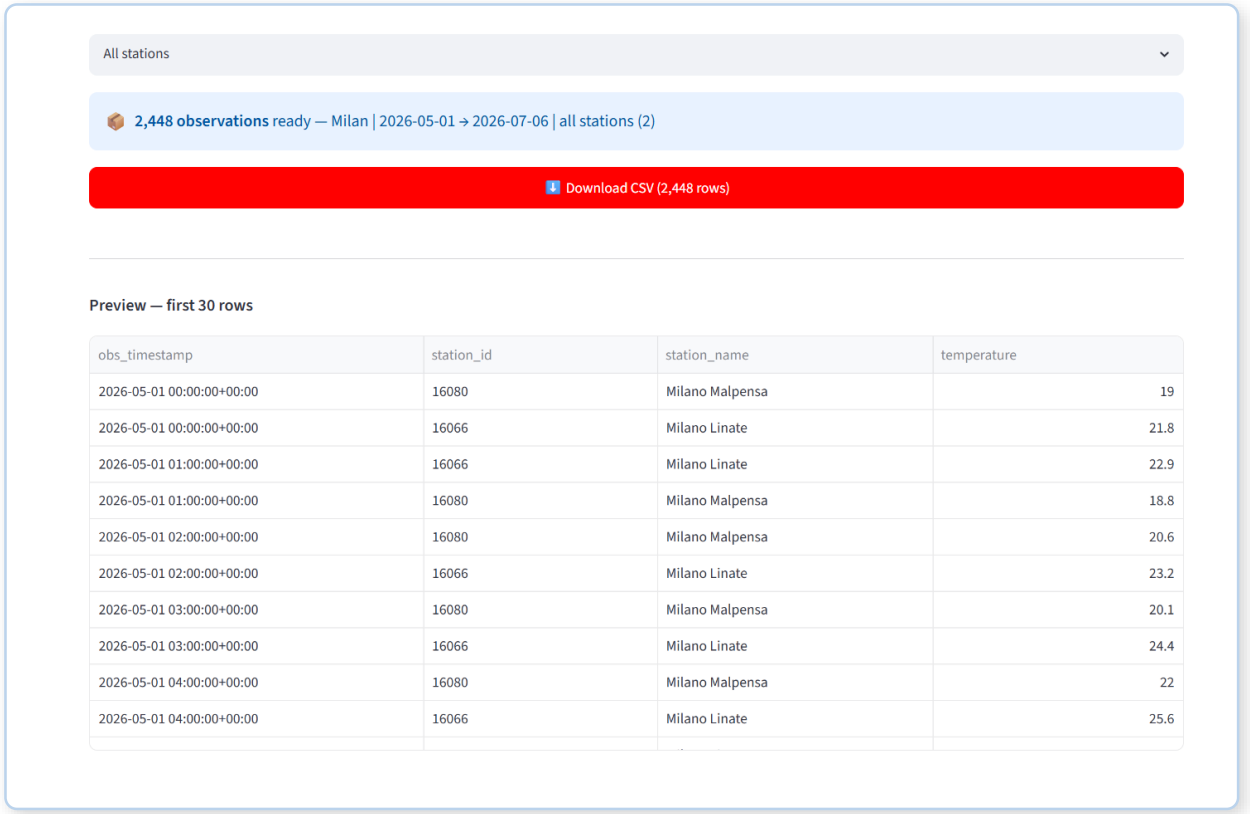


Aggregated bars and histograms summarise repeated patterns across the selected period.

Export CSV — From Output to Ownership

The export tab confirms how many observations are ready and lets the user download the filtered dataset.

Every role can export.
Students, NGOs, and Government Officials all have export access — turning the dashboard's filtered view into a portable dataset.



obs_timestamp	station_id	station_name	temperature
2026-05-01 00:00:00+00:00	16080	Milano Malpensa	19
2026-05-01 00:00:00+00:00	16066	Milano Linate	21.8
2026-05-01 01:00:00+00:00	16066	Milano Linate	22.9
2026-05-01 01:00:00+00:00	16080	Milano Malpensa	18.8
2026-05-01 02:00:00+00:00	16080	Milano Malpensa	20.6
2026-05-01 02:00:00+00:00	16066	Milano Linate	23.2
2026-05-01 03:00:00+00:00	16080	Milano Malpensa	20.1
2026-05-01 03:00:00+00:00	16066	Milano Linate	24.4
2026-05-01 04:00:00+00:00	16080	Milano Malpensa	22
2026-05-01 04:00:00+00:00	16066	Milano Linate	25.6